



Лекція 8

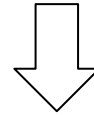
Advanced Spring Boot

**OpenAPI. Spring Data. Liquibase.
Schedulers. Cache. AOP. Logging**

banner.txt

```
      _ _ _ _ _      _ _ _ _ _      _ _ _ _ _  
  /\ / _ _ _ ' _ _ _ _ _ ( ) _ _ _ _ _ _ _ _ _  
( ( ) \ _ _ _ | ' | ' | ' | ' | ' | ' | ' | ' |  
  \W _ _ _ | | | | | | | | | | | | | | | | | | |  
  ' | _ _ _ | _ _ | | | | | | | | | | | | | | |  
  =====|_|=====| _ _ _ / _ _ _ / _ _ _ /
```

:: Spring Boot :: (v3.5.8)



Banner generator

<https://devops.datenkollektiv.de/banner.txt/index.html>

OpenAPI docs and Swagger UI

pom.xml

```
<!-- OpenAPI Documentation -->
```

```
<dependency>
```

```
    <groupId>org.springdoc</groupId>
```

```
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
```

```
    <version>2.8.14</version>
```

```
</dependency>
```

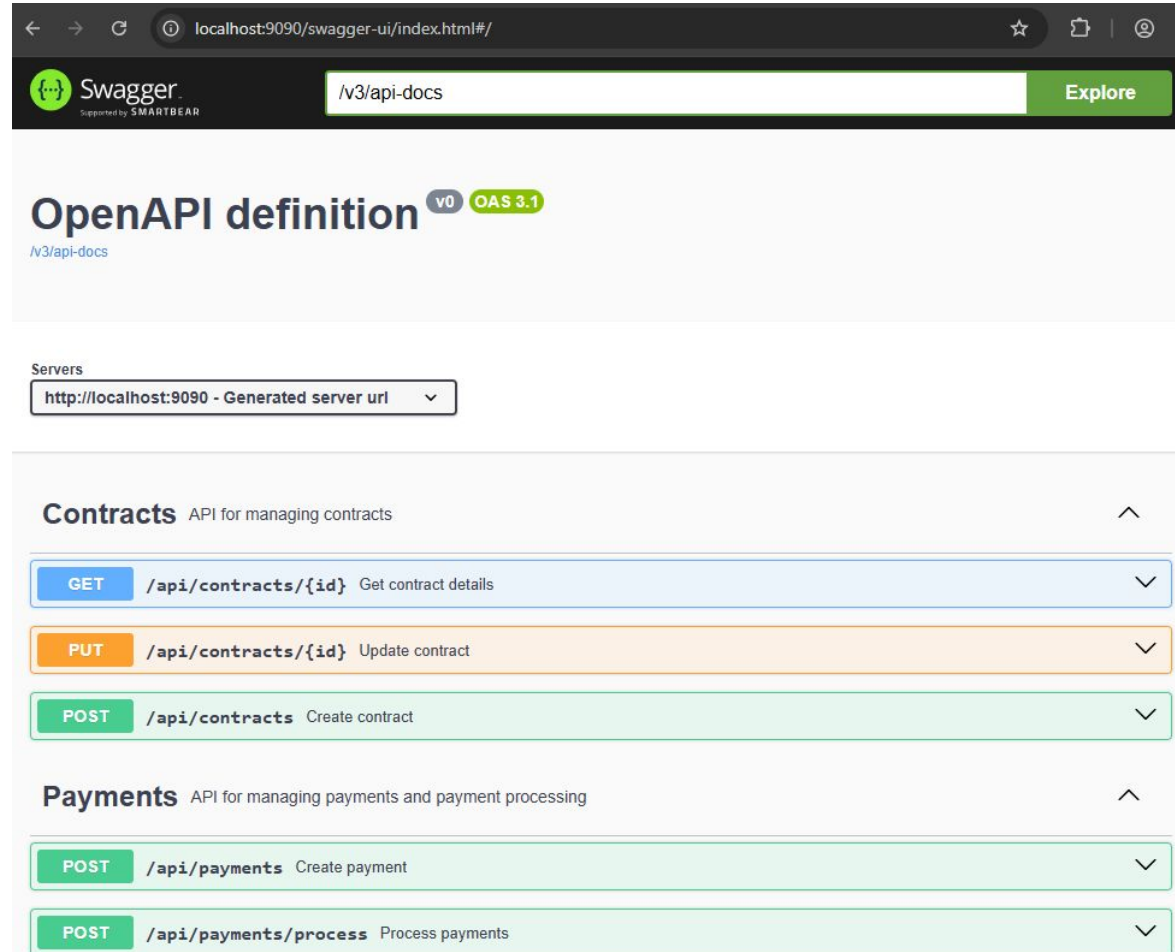
application.properties

```
# OpenAPI Documentation
```

```
springdoc.swagger-ui.enabled=true
```

<https://www.baeldung.com/spring-rest-openapi-documentation>

Swagger UI



The image shows a web browser window displaying the Swagger UI. The address bar shows the URL `localhost:9090/swagger-ui/index.html#`. The Swagger logo and "Supported by SMARTBEAR" are visible. A search bar contains `/v3/api-docs` and an "Explore" button. The main heading is "OpenAPI definition" with a "v0" tag and an "OAS 3.1" badge. Below this is a "Servers" section with a dropdown menu showing `http://localhost:9090 - Generated server url`. The "Contracts" section is expanded, showing three endpoints: GET `/api/contracts/{id}` (Get contract details), PUT `/api/contracts/{id}` (Update contract), and POST `/api/contracts` (Create contract). The "Payments" section is also expanded, showing two endpoints: POST `/api/payments` (Create payment) and POST `/api/payments/process` (Process payments).

Swagger
Supported by SMARTBEAR

`/v3/api-docs` Explore

OpenAPI definition v0 OAS 3.1

[/v3/api-docs](#)

Servers

`http://localhost:9090 - Generated server url`

Contracts API for managing contracts

- GET** `/api/contracts/{id}` Get contract details
- PUT** `/api/contracts/{id}` Update contract
- POST** `/api/contracts` Create contract

Payments API for managing payments and payment processing

- POST** `/api/payments` Create payment
- POST** `/api/payments/process` Process payments

OpenAPI Annotations

```
@RestController
@RequestMapping("/api/contracts")
@Tag(name = "Contracts", description = "API for managing contracts")
public class ContractController {

    @Operation(summary = "Get contract details", description = "Retrieves contract details by contract ID")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Contract found",
            content = @Content(mediaType = "application/json", schema = @Schema(implementation =
ContractDetailsDto.class))),
        @ApiResponse(responseCode = "404", description = "Contract not found",
            content = @Content(mediaType = "application/json", schema = @Schema(implementation =
ProblemDetail.class)))
    })
    @GetMapping("/{id}")
    public ContractDetailsDto getContractDetails(
        @Parameter(description = "Contract ID", required = true) @PathVariable("id") String id) {
        return contractService.getContractDetails(id);
    }
}
```

Spring Data JPA

Spring Data JPA makes it easy to easily implement JPA-based (Java Persistence API) repositories.

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```

<https://spring.io/projects/spring-data-jpa>

JpaRepository

@Repository

public interface ContractRepository

extends JpaRepository<ContractData, String> {

Optional<ContractData> findByNumber(String number);

Stream<ContractData> findAllBySignDateBetween(LocalDate from, LocalDate to);

@Query("SELECT id FROM ContractData WHERE status = ?1")

List<String> findIdsByStatus(ContractStatus status);

}

// all such code should be covered by integration tests

Liquibase

Liquibase is an open-source database schema change management tool that allows tracking, managing, and applying database schema changes across multiple environments and databases through version-controlled XML, JSON, YAML, or SQL formats.

pom.xml:

```
<dependency>
```

```
    <groupId>org.liquibase</groupId>
```

```
    <artifactId>liquibase-core</artifactId>
```

```
    <version>4.27.0</version>
```

```
</dependency>
```

application.properties:

```
spring.liquibase.change-log=classpath:liquibase-changelog.xml
```

liquibase-changelog.xml

```
<databaseChangeLog xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

    xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-latest.xsd" >

    <changeSet id="create-schema" author="profitsoft">
        <createTable tableName="contract">
            <column name="id" type="VARCHAR(64)">
                <constraints primaryKey="true" nullable="false"/>
            </column>
            <column name="number" type="VARCHAR(255)">
                <constraints nullable="false" unique="true"
uniqueConstraintName="contract_number_unique" />
            </column>
            ...
        </createTable>
    </changeSet>
</databaseChangeLog>
```

Dates and Time in Databases and REST API

Instant - when you need date and time

LocalDate - when you need only date (like birth date)

In DB store in **UTC**.

In REST API automatically converted according to [ISO 8601](#) format.

```
"signDate": "2024-04-23"
```

```
"operationTime": "2024-04-23T20:15:05.185697Z"
```

On UI convert according to user's Time Zone or specify explicitly that it's UTC.

Scheduling

Schedulers allow to run background processes at specified time.

@EnableScheduling

@SpringBootApplication

```
public class SpringAdvancedApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(SpringAdvancedApplication.class, args);  
    }  
}
```

Scheduler

@Slf4j

@Component

@RequiredArgsConstructor

```
public class PaymentScheduler {
```

```
    private final PaymentService paymentService;
```

```
    /**
```

```
     * Runs at second :00 every 5 minutes starting at minute :00 every hour.
```

```
    */
```

```
    @Scheduled(cron = "${payment.processing.cron:0 0/5 * * * ?}")
```

```
    public void processPayments() {
```

```
        try {
```

```
            paymentService.processPayments();
```

```
        } catch (Throwable e) {
```

```
            log.error("Error processing payments", e);
```

```
        }
```

```
    }
```

```
}
```

SchedLock

SchedLock is Java library that makes sure our scheduled tasks run only once at the same time.

```
@Configuration
@EnableScheduling
@ConditionalOnProperty(name = "scheduling.enabled",
                        havingValue = "true", matchIfMissing = true)
@EnableSchedulerLock(defaultLockAtMostFor = "PT1M")
public class SchedulingConfig {

    @Bean
    public LockProvider lockProvider(DataSource dataSource) {
        return new JdbcTemplateLockProvider(dataSource);
    }
}
```

<https://www.baeldung.com/shedlock-spring>

@SchedulerLock

```
@Scheduled(cron = "${payment.processing.cron:0 0/5 * * * ?}")
@SchedulerLock(name = "ProcessPayments",
               lockAtLeastFor = "PT30S", lockAtMostFor = "PT10M")
public void processPayments() {
    try {
        paymentService.processPayments();
    } catch (Throwable e) {
        log.error("Error processing payments", e);
    }
}
```

Batch Processing Principles

- Avoid loading too many (or all) objects into memory at once
 - Process data in batches
 - typically 100–1000 items, depending on their size
- Minimize the number of database calls
 - ideally, a single query per batch
- Assume the process can be interrupted at any time (server restart, etc.)
 - Manage transactions
 - What happens when it starts again?
- Structure the processing logic in code
 - Separate method for a “batch”
 - Separate method for a single record
 - Methods return processing results (statistics, etc.)
- Log the processing flow

Caching

Caching is the process of storing frequently accessed data or computational results in a readily accessible location to speed up future requests for the same information.

- **Local (in-process) cache**
 - Stored in the memory of a single application instance
 - Ultra-low latency, no network hop
 - Not shared between instances → possible inconsistencies across nodes
 - Good for per-request/session data, config, small reference datasets
- **Centralized (distributed) cache**
 - Shared cache service (e.g., Redis, Memcached) accessed over the network
 - Consistent view of cached data across multiple application instances
 - Slightly higher latency than local, but reduces load on databases and APIs
 - Good for shared domain data, expensive queries, cross-service reuse

Spring Boot + Ehcache

Ehcache is an open source, standards-based cache that boosts performance, offloads your database, and simplifies scalability.

pom.xml:

```
<dependency>  
    <groupId>org.ehcache</groupId>  
    <artifactId>ehcache</artifactId>  
    <version>3.10.8</version>  
    <classifier>jakarta</classifier>  
</dependency>
```

application.properties:

```
spring.cache.jcache.config=classpath:ehcache.xml
```

```
@EnableCaching
```

```
@Cacheable(value = "CONTRACT", key = "#number")
```

```
@CacheEvict("CONTRACT")
```

ehcache.xml

```
<config
  xmlns:xsi='http://www.w3.org/2001/XMLSchema-instance'
  xmlns='http://www.ehcache.org/v3'
  xmlns:ehcache='http://www.ehcache.org/v3/jsr107'>

  <cache alias="CONTRACT" uses-template="short-cache" />

  <cache-template name="short-cache">
    <expiry>
      <ttl unit="minutes">1</ttl>
    </expiry>
    <resources>
      <heap unit="entries">100</heap>
    </resources>
  </cache-template>
</config>
```



Aspect Oriented Programming

AOP is a programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns, such as logging and transaction management, through the definition of aspects that can be applied across various points of an application independently from the main business logic.

- **Aspect** is a module that encapsulates behaviors that cut across multiple classes, such as transaction management or logging. Aspects contain advice and may also include metadata, such as declarations for pointcuts.
- **Join Point** is a specific point in the execution of the program, such as method execution, exception handling, or field access. It represents a point in the application where an aspect can be applied.

Key AOP Concepts

- **Advice** defines the action to be taken by an aspect at a particular join point. Different types of advice include "before", "after", and "around" advice, which determine when the code in the aspect is executed in relation to the join point.
- **Pointcut** defines the criteria for selecting join points of interest. It essentially filters out certain join points based on some conditions, specifying where the associated advice should be applied.
- **Target Object** is the object being advised by one or more aspects. It is the object on which the business logic methods are executed.

Aspect Example

```
@Slf4j
@Aspect
@Component
public class ServiceMonitor { // Aspect

    @Around( // Advice
        "@annotation(Monitored)" // Pointcut
    ) // v Join Point v
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        joinPoint.getTarget(); // Target Object
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        log.debug("Method '{}' with params={} executed in {} ms",
            joinPoint.getSignature().toShortString(), List.of(joinPoint.getArgs()),
            System.currentTimeMillis() - start);
        return result;
    }
}
```



Logging

Logging is the process of recording information about a program's execution to help Developers and Ops understand the program's flow, diagnose issues, and monitor performance at runtime.

Logging levels

- trace
- debug - dev/test default and when needed on prod
- info - production default
- warn
- error

Logback

Spring Boot uses **Logback** for logging by default.

```
# Enables DEBUG logging for classes in the dev.profitsoft.fd package  
logging.level.dev.profitsoft.fd=DEBUG
```

@Slf4j annotation is used to inject *log* attribute into a class.

You may add `logback.xml` file and configure fine-grained logging there

<https://www.baeldung.com/spring-boot-logging>

Integration Tests

application-test.properties:

spring.datasource.url=jdbc:h2:mem:myDb;DB_CLOSE_DELAY=-1

scheduling.enabled=false

```
@SpringBootTest
```

```
@ActiveProfiles("test")
```

```
class PaymentServiceTest {
```

```
    ...
```

```
}
```

Sample Demo App

<https://github.com/profitsoft-jfd/advanced-spring>